LAPORAN PRAKTIKUM BASIS DATA
UNIT 6, 8, DAN 9
GROUP FUNCTIONS DAN *JOINS*

Disusun oleh:

Nama	: Azura Rizky Annisa
NIM	: 24/535715/SV/24290
Kelas	: B1
Dosen Pengampu	: Anni Karimatul Fauziyyah, S.Kom., M.Eng.

PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA INTERNET
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
2025

UNIT 6, 8, DAN 9

GROUP FUNCTIONS DAN *JOINS*

A. Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut :

1. Mahasiswa mampu memahami pengertian *Single Raw Function* pada SQL.
2. Mahasiswa mampu memahami pengertian *JOINs* pada SQL.
3. Mahasiswa mampu menerapkan *JOINs* pada SQL

B. Dasar Teori

Fungsi agregasi dalam SQL seperti COUNT(), SUM(), AVG(), MAX(), dan MIN() digunakan untuk melakukan perhitungan pada sekelompok data dan menghasilkan satu nilai ringkasan. Fungsi-fungsi ini sering dipadukan dengan klausa GROUP BY yang berperan mengelompokkan data berdasarkan kolom tertentu, misalnya menghitung jumlah karyawan per departemen. Untuk melakukan penyaringan hasil agregasi, digunakan klausa HAVING yang berfungsi mirip WHERE namun bekerja setelah proses pengelompokan, contohnya menampilkan departemen dengan rata-rata gaji di atas 5000. Perlu dipahami bahwa WHERE menyaring data sebelum pengelompokan, sedangkan HAVING menyaring setelahnya.

Di sisi lain, operasi join memungkinkan penggabungan data dari beberapa tabel berdasarkan relasi antar kolom. INNER JOIN menampilkan hanya baris yang cocok di kedua tabel, sementara LEFT JOIN mengambil semua data dari tabel pertama meskipun tidak ada kecocokan di tabel kedua. RIGHT JOIN bekerja kebalikan dari LEFT JOIN, dan FULL JOIN menggabungkan semua baris dari kedua tabel. Ada juga CROSS JOIN yang menghasilkan kombinasi semua baris serta SELF JOIN yang berguna untuk relasi dalam satu tabel, seperti menghubungkan data karyawan dengan manajernya. Pemahaman mendalam tentang berbagai jenis join ini sangat penting untuk pengambilan data yang efektif dari database relasional yang kompleks. Kedua konsep dasar SQL ini - fungsi agregasi dan operasi join - merupakan fondasi penting dalam manipulasi dan analisis data.

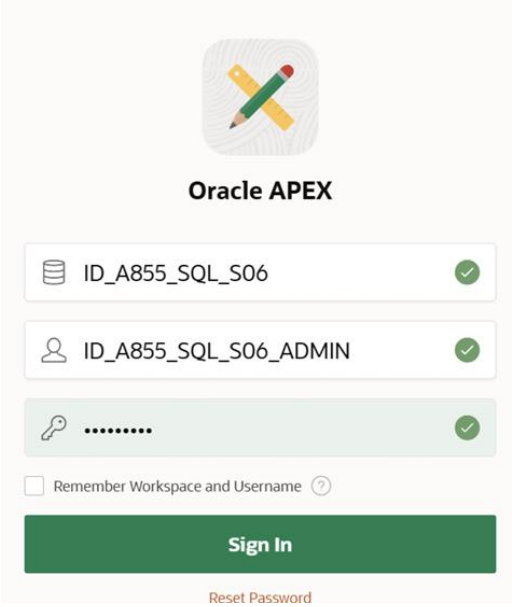
C. Alat dan Bahan

Alat dan Bahan yang digunakan pada praktikum ini adalah sebagai berikut:

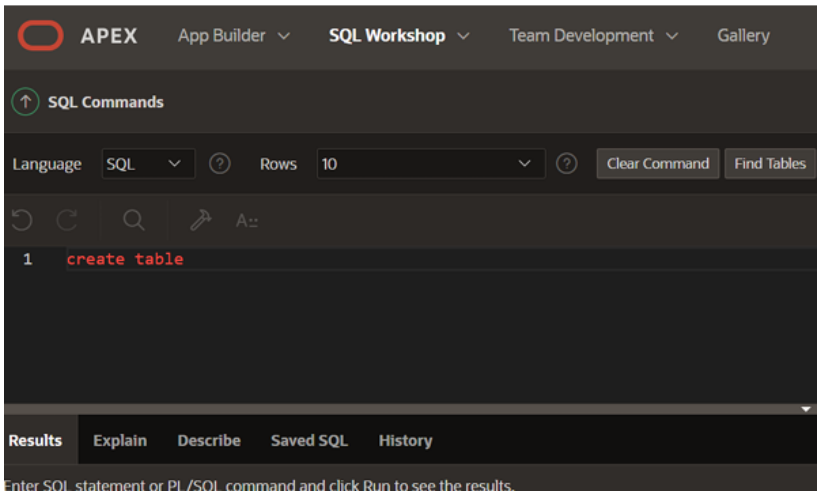
1. Laptop/Desktop/Komputer
2. Oracle SQL APEX

D. Prosedur Percobaan

1. Buka website https://iacademy2.oracle.com
2. Login ke akun masing-masing



3. Pilih fitur SQL Commands dan ketik perintahnya



E. Hasil dan Pembahasan

DP 8-1: Group Functions

1. Hal 13

```
1  SELECT MIN(hire_date)
2  FROM employees;
3
```

Results | Explain | Describe | Saved SQL | History

MIN(HIRE_DATE)
17-Jun-1987

1 rows returned in 0.02 seconds Download

```
1  SELECT MIN(country_name)
2  FROM wf_countries;
```

Results | Explain | Describe | Saved SQL | History

MIN(COUNTRY_NAME)
Anguilla

1 rows returned in 0.00 seconds Download

```
1  SELECT MIN(life_expect_at_birth)
2  AS "Lowest Life Exp"
3  FROM wf_countries;
```

Results | Explain | Describe | Saved SQL | History

Lowest Life Exp
32.62

1 rows returned in 0.02 seconds Download

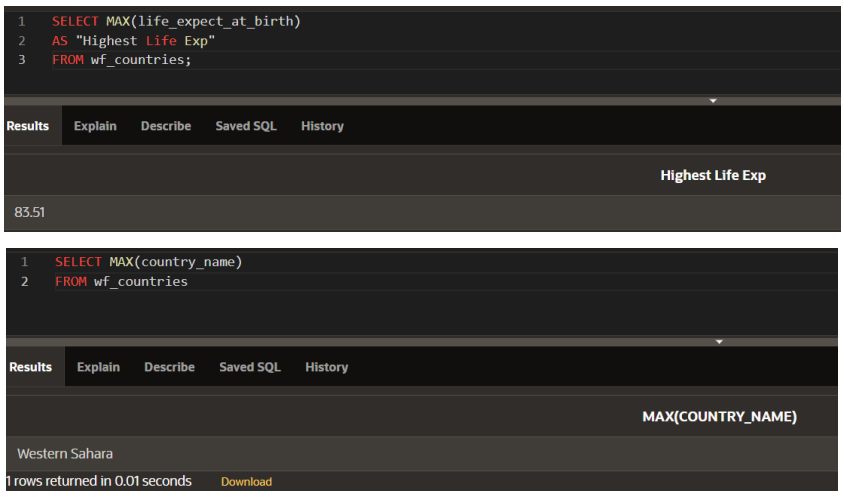
Pembahasan :

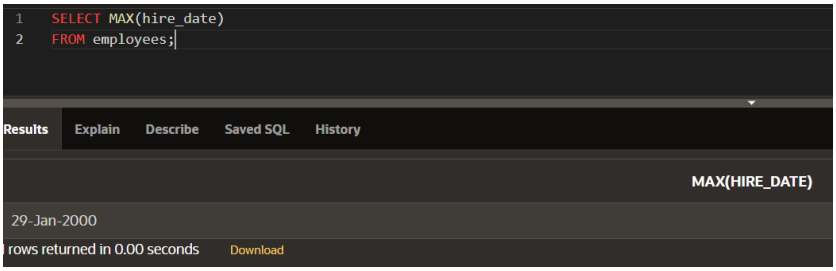
Kode pada gambar pertama menunjukkan penggunaan fungsi agregat MIN untuk mencari nilai terkecil dari kolom hire_date dalam tabel employees. Fungsi MIN mengembalikan tanggal perekrutan paling awal dari semua karyawan yang tercatat dalam tabel tersebut. Hasil query menunjukkan bahwa tanggal perekrutan paling awal adalah 17 Juni 1987, yang berarti karyawan pertama dalam tabel tersebut direkrut pada tanggal tersebut. Query ini berguna untuk memahami sejarah perekrutan karyawan dalam suatu organisasi.

Pada gambar kedua, kode tersebut menggunakan fungsi MIN untuk mencari nilai terkecil dari kolom country_name dalam tabel wf_countries. Fungsi MIN ketika diterapkan pada kolom teks akan mengembalikan nilai pertama berdasarkan urutan abjad (lexicographical order). Hasil query menunjukkan bahwa negara dengan nama terawal dalam urutan abjad adalah "Anguilla". Query semacam ini dapat digunakan untuk keperluan pengurutan atau analisis data geografis.

Kode pada gambar ketiga menggunakan fungsi MIN untuk mencari nilai terkecil dari kolom life_expect_at_birth dalam tabel wf_countries, dengan memberikan alias "Lowest Life Exp" pada hasilnya. Fungsi MIN mengembalikan angka harapan hidup terendah yang tercatat dalam tabel, yaitu 32.62 tahun. Hasil ini memberikan insight tentang negara dengan kondisi kesehatan atau kesejahteraan yang paling rendah dalam dataset tersebut. Penggunaan alias membuat hasil query lebih mudah dibaca dan dipahami.

2. Hal 14





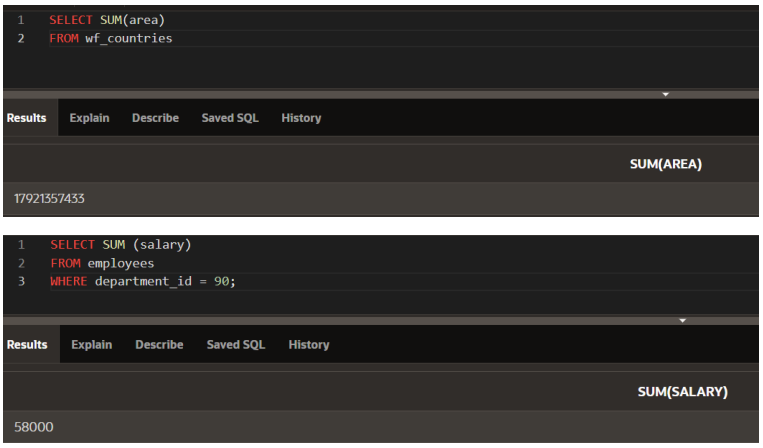
Pembahasan :

Kode menggunakan fungsi agregat MAX untuk menemukan tanggal terbesar di kolom hire_date dari tabel employees, yaitu 29 Januari 2000. Ini menunjukkan rekrutmen terakhir perusahaan dan berguna untuk analisis tren perekrutan.

Fungsi MAX diterapkan pada kolom teks country_name dalam tabel wf_countries, mengembalikan nilai terakhir secara abjad ("Western Sahara"). Query ini bermanfaat untuk pengurutan atau penyaringan data.

Fungsi MAX mencari nilai tertinggi di kolom life_expect_at_birth (83.51 tahun) dari tabel wf_countries, dengan alias "Highest Life Exp" untuk memudahkan interpretasi. Hasilnya mencerminkan negara dengan harapan hidup terbaik dalam dataset.

3. Hal 15



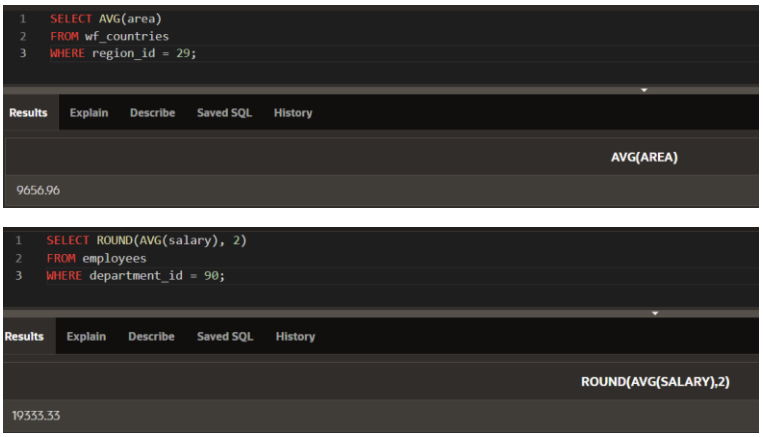
Pembahasan :

Kode pada gambar pertama menerapkan fungsi agregat SUM guna menghitung jumlah total gaji (salary) karyawan di departemen dengan department_id = 90. Fungsi SUM bekerja dengan menjumlahkan seluruh nilai gaji dalam kolom yang memenuhi kriteria WHERE. Hasil kueri mengungkapkan bahwa total gaji departemen 90 adalah

58.000, sebuah informasi yang berguna untuk evaluasi pengeluaran departemen atau penyusunan anggaran. Kueri semacam ini membantu dalam menganalisis distribusi gaji di suatu organisasi berdasarkan departemen tertentu.

Sementara itu, kode pada gambar kedua memanfaatkan fungsi SUM untuk menghitung total luas wilayah (area) dari seluruh negara dalam tabel wf_countries. Fungsi ini menghasilkan penjumbahan semua nilai di kolom area, yakni 17.921.357.433 (dalam satuan luas, seperti kilometer persegi). Data ini memberikan insight mengenai total cakupan geografis dalam dataset. Kueri seperti ini umum dipakai untuk keperluan analisis statistik atau perbandingan data geografis dalam skala besar.

4. Hal 16



Pembahasan :

Kode di gambar pertama memanfaatkan fungsi STDDEV (standar deviasi) guna mengevaluasi sebaran data pada kolom life_expect_at_birth dalam tabel wf_countries, dengan hasil yang dibulatkan hingga empat desimal menggunakan fungsi ROUND. Standar deviasi menggambarkan tingkat penyimpangan data dari nilai rata-ratanya. Hasil kueri, yakni 11.9683, menunjukkan bahwa harapan hidup di berbagai negara memiliki variasi yang cukup besar. Kueri semacam ini bermanfaat dalam analisis statistik untuk mengevaluasi kesenjangan tingkat kesehatan atau kesejahteraan antarnegara.

Sementara itu, kode pada gambar kedua mengaplikasikan fungsi VARIANCE (varian) untuk mengukur dispersi data dari kolom life_expect_at_birth, dengan pembulatan hingga empat angka desimal. Varian merupakan kuadrat dari standar deviasi dan memberikan pemahaman lebih mendalam tentang sebaran data. Hasil kueri,

yaitu 143.2394, mengindikasikan adanya disparitas yang cukup tinggi dalam angka harapan hidup di berbagai negara. Kueri semacam ini kerap dipakai dalam penelitian statistik untuk mengidentifikasi ketidakmerataan atau pola tertentu dalam suatu dataset.

5. Hal 17





Pembahasan :

Kode awal menerapkan fungsi STDDEV() untuk mengukur dispersi data harapan hidup (life_expect_at_birth), dengan presisi empat digit desimal melalui fungsi ROUND(). Angka standar deviasi sebesar 11.9683 tahun mengungkapkan variasi yang substansial dalam angka harapan hidup antarnegara. Besarnya nilai ini (mendekati 12 tahun) menyoroti ketimpangan yang nyata, yang mungkin dipengaruhi oleh disparitas dalam sistem kesehatan, tingkat ekonomi, atau kondisi lingkungan di masing-masing negara. Analisis semacam ini penting untuk mengevaluasi ketidakmerataan kesehatan secara kuantitatif pada skala global.

Kode berikutnya menggunakan VARIANCE() pada dataset yang sama, juga dengan pembulatan empat desimal. Nilai varians 143.2394 - yang secara matematis merupakan kuadrat dari standar deviasi sebelumnya - memberikan perspektif lebih sensitif tentang distribusi data. Besarnya angka varians ini semakin mengkonfirmasi adanya heterogenitas ekstrim dalam statistik harapan hidup dunia. Kedua pengukuran statistik ini saling melengkapi, membentuk landasan analitik yang kokoh untuk: (1) mengeksplorasi determinan kesenjangan kesehatan global, dan (2) menyediakan kerangka kerja untuk studi longitudinal tentang perkembangan kesehatan masyarakat internasional.

6. Hal 18

```
SELECT AVG(commission_pct)
FROM employees;
```

Results | Explain | Describe | Saved SQL | History

AVG(COMMISSION_PCT)
.2125

Pembahasan :

Kode tersebut menggunakan fungsi agregat AVG() untuk menghitung nilai rata-rata dari kolom commission_pct (persentase komisi) dalam tabel employees. Hasil query menunjukkan bahwa rata-rata persentase komisi karyawan adalah 0.2125 atau 21.25%. Nilai ini memberikan gambaran umum tentang struktur kompensasi berbasis komisi di perusahaan tersebut. Query semacam ini berguna untuk analisis kebijakan kompensasi, perencanaan anggaran, atau evaluasi kinerja sistem insentif karyawan. Hasil rata-rata ini juga dapat menjadi acuan untuk membandingkan komisi antar departemen atau mengevaluasi apakah kebijakan komisi saat ini sudah efektif.

7. Hal 19

```
SELECT MAX(salary), MIN(salary), MIN(employee_id)
FROM employees
WHERE department_id = 60;
```

Results | Explain | Describe | Saved SQL | History

MAX(SALARY)	MIN(SALARY)	MIN(EMPLOYEE_ID)
9000	4200	103

Pembahasan :

Kode SQL ini menggunakan fungsi agregat MAX() dan MIN() untuk mengambil informasi gaji karyawan di departemen dengan ID 68. Hasilnya menunjukkan gaji tertinggi sebesar 9.000 dan gaji terendah 4.200, dengan selisih cukup besar yaitu 4.800. Selain itu juga ditampilkan ID karyawan terendah yaitu 103 yang kemungkinan merupakan karyawan pertama di departemen tersebut.

Data ini berguna untuk menganalisis struktur gaji dan pola rekrutmen di departemen 68. Rentang gaji yang lebar bisa menunjukkan perbedaan level jabatan atau masa kerja. Sementara ID karyawan terendah memberi gambaran tentang sejarah rekrutmen. Informasi seperti ini penting untuk evaluasi kebijakan SDM, perencanaan penggajian, dan analisis perkembangan tim dalam departemen.

DP 8-2: COUNT, DISTINCT, NVL

1. Hal 5

```
1  SELECT COUNT(job_id)
2  FROM employees;
3
```

Results | Explain | Describe | Saved SQL | History

COUNT(JOB_ID)
20

Pembahasan :

Kode tersebut menggunakan fungsi COUNT() untuk melakukan perhitungan jumlah entri pada kolom job_id dalam tabel employees, dengan hasil query menunjukkan terdapat 20 job_id yang tercatat. Fungsi ini secara khusus hanya menghitung baris dimana job_id tidak bernilai NULL. Data ini memberikan gambaran mengenai banyaknya posisi pekerjaan yang sedang aktif dalam perusahaan, namun interpretasinya memerlukan kehati-hatian karena angka tersebut bisa merepresentasikan dua kemungkinan: jumlah karyawan (dengan asumsi satu karyawan memiliki satu job_id) atau sekadar jumlah entri job_id (termasuk yang duplikat). Untuk analisis yang lebih tepat, disarankan menggunakan COUNT(employee_id) apabila ingin mengetahui jumlah total karyawan, atau COUNT(DISTINCT job_id) jika tujuan utamanya adalah mengidentifikasi jumlah jenis pekerjaan unik yang ada. Pemahaman terhadap perbedaan ini penting untuk memastikan analisis data yang akurat dan sesuai dengan kebutuhan informasi yang diinginkan.

2. Hal 7

```
2  FROM employees
3  WHERE hire_date < '01-Jan-1996';
4
```

Results | Explain | Describe | Saved SQL | History

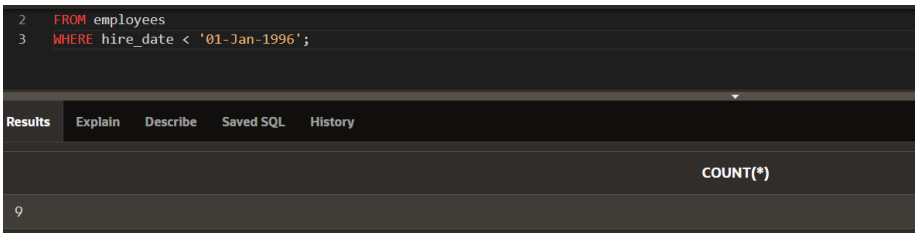
COUNT(*)
9

Pembahasan :

Analisis ini menerapkan fungsi COUNT(*) untuk mengidentifikasi jumlah karyawan dalam tabel employees yang bergabung dengan perusahaan sebelum 1 Januari 1996, dengan menggunakan filter WHERE hire_date < '01-Jan-1996'. Hasil pencarian mengungkapkan terdapat 9 orang karyawan yang termasuk dalam kategori ini. Data semacam ini memiliki nilai strategis bagi departemen SDM karena dapat

membantu dalam: (1) pemetaan karyawan berpengalaman (senior), (2) perencanaan program pensiun, serta (3) evaluasi retensi karyawan jangka panjang. Informasi tentang karyawan dengan masa kerja lebih dari 25 tahun ini juga berguna untuk penyusunan kebijakan pengembangan karir dan pemberian penghargaan kepada loyalitas karyawan. Dengan memahami profil karyawan senior tersebut, perusahaan dapat membuat keputusan yang lebih tepat dalam mengelola talenta dan menjaga pengetahuan organisasi.

3. Hal 8



Pembahasan :

Fungsi COUNT(*) dalam kode ini berperan untuk menghitung jumlah karyawan dalam tabel employees yang bergabung dengan perusahaan sebelum tanggal 1 Januari 1996, dengan menggunakan kriteria hire_date < '01-Jan-1996'. Berdasarkan hasil query, ditemukan bahwa terdapat 9 orang karyawan yang termasuk dalam kelompok ini. Analisis semacam ini memiliki nilai strategis bagi perusahaan karena dapat diaplikasikan dalam berbagai konteks, mulai dari evaluasi historis proses rekrutmen hingga identifikasi karyawan berpengalaman yang telah mengabdi lebih dari 28 tahun (hingga tahun 2024). Dari perspektif manajemen SDM, data ini dapat dimanfaatkan untuk menyusun program pensiun yang terencana maupun merancang sistem penghargaan bagi karyawan berdedikasi tinggi. Pada aspek teknis, perlu diperhatikan bahwa format tanggal yang digunakan mengacu pada konvensi Oracle SQL, sementara operator < berfungsi untuk menyaring data karyawan yang tanggal bergabungnya lebih awal dari tanggal patokan yang ditetapkan.

4. Hal 10

```
1   SELECT DISTINCT job_id
2   FROM employees;
```

Results | Explain | Describe | Saved SQL | History

JOB_ID
AC_ACCOUNT
AC_MGR
AD_ASST
AD_PRES
AD_VP
IT_PROG
MK_MAN
MK_REP
SA_MAN
SA_REP

More than 10 rows available. Increase rows selector to view more rows.

Pembahasan :

Analisis ini menerapkan klausa SELECT DISTINCT pada kolom job_id dari tabel employees untuk mengekstrak nilai-nilai unik yang merepresentasikan berbagai posisi dalam perusahaan. Hasil query menampilkan beragam jabatan seperti 'AC_ACCOUNT', 'AC_MGR', 'AD_ASST', 'AD_PRES', dan 'AD_VP' tanpa adanya duplikasi data. Pendekatan ini memberikan beberapa manfaat penting bagi manajemen SDM: (1) memetakan struktur organisasi secara jelas dengan mengidentifikasi semua peran yang ada, (2) menyajikan dasar data yang akurat untuk analisis distribusi sumber daya manusia antar jabatan, dan (3) mendukung perencanaan strategis pengembangan organisasi. Dengan menampilkan setiap jenis job_id hanya sekali, teknik query ini memungkinkan perusahaan untuk memperoleh gambaran komprehensif tentang keragaman posisi yang tersedia, sekaligus memfasilitasi evaluasi terhadap struktur jabatan yang ada dalam organisasi.

5. Hal 12

```
1   SELECT SUM(salary)
2   FROM employees
3   WHERE department_id = 90;
```

Results | Explain | Describe | Saved SQL | History

SUM(SALARY)
58000

```
1   SELECT SUM(DISTINCT salary)
2   FROM employees
3   WHERE department_id = 90;
```

Results | Explain | Describe | Saved SQL | History

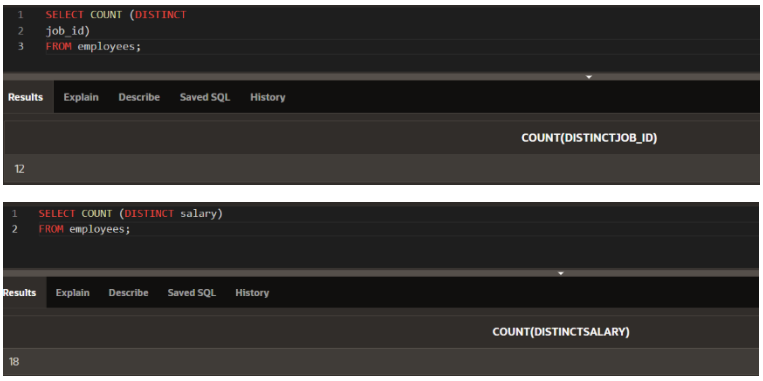
SUM(DISTINCTSALARY)
41000

Pembahasan :

Kode pertama menerapkan fungsi SUM(salary) untuk menghitung total pengeluaran gaji di departemen dengan department_id = 90, menghasilkan nilai 58.000. Angka ini mencakup seluruh pembayaran gaji, termasuk yang bernilai sama (duplikat), sehingga merepresentasikan beban gaji riil departemen tersebut. Query semacam ini sangat berguna untuk keperluan analisis keuangan, seperti perencanaan anggaran departemen atau perbandingan efisiensi penggajian antar divisi.

Sementara itu, kode kedua menggunakan SUM(DISTINCT salary) untuk menghitung total gaji unik (hanya nilai berbeda) pada departemen yang sama. Meskipun hasil spesifik tidak ditampilkan (mungkin karena semua gaji sama atau terdapat kesalahan teknis), fungsi ini berguna untuk menganalisis variasi struktur kompensasi tanpa terpengaruh redundansi data.

6. Hal 13



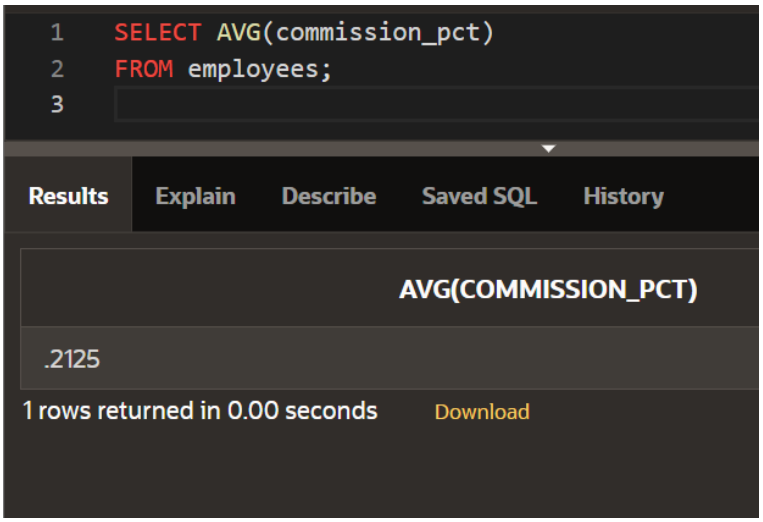
Pembahasan :

Analisis data karyawan mengungkapkan dua aspek penting dalam struktur organisasi perusahaan. Pertama, penggunaan COUNT(DISTINCT salary) menunjukkan terdapat 18 nilai gaji berbeda di antara seluruh karyawan, yang mencerminkan variasi sistem kompensasi yang diterapkan. Angka ini mengindikasikan bahwa perusahaan memiliki struktur penggajian yang cukup terdiferensiasi, di mana beberapa karyawan mungkin menerima gaji yang sama sementara lainnya memiliki tingkat kompensasi berbeda. Kedua, hasil COUNT(DISTINCT job_id) yang mencatat 12 jabatan unik mengungkapkan keragaman peran dan spesialisasi dalam organisasi. Data-data ini secara bersama memberikan wawasan berharga bagi manajemen dalam mengevaluasi kebijakan SDM,

mulai dari analisis kesetaraan gaji, identifikasi kebutuhan pelatihan spesifik, hingga perencanaan pengembangan struktur organisasi yang lebih efektif. Kedua metrik tersebut saling melengkapi dalam menggambarkan kompleksitas sistem kompensasi dan diferensiasi peran kerja di perusahaan.

7. Hal 15





Pembahasan :

Kode pertama menggunakan fungsi NVL(commission_pct, 0) dalam perhitungan AVG() untuk mengganti nilai NULL pada kolom komisi dengan 0 sebelum dihitung rata-ratanya. Hasil 0.0425 (4.25%) menunjukkan rata-rata komisi yang lebih rendah dibanding kode kedua, karena karyawan tanpa komisi (NULL) dihitung sebagai 0. Pendekatan ini memberikan gambaran lebih akurat tentang kebijakan komisi perusahaan secara keseluruhan, termasuk karyawan yang tidak mendapatkan komisi.

Teknik NVL sangat berguna ketika NULL perlu diartikan sebagai nilai netral dalam perhitungan statistik.

Kode kedua menghitung AVG(commission_pct) secara langsung tanpa mengubah nilai NULL, menghasilkan rata-rata 0.2125 (21.25%). Nilai yang lebih tinggi ini hanya memperhitungkan karyawan yang benar-benar memiliki komisi (non-NULL), sehingga tidak mencerminkan seluruh populasi karyawan. Perbedaan signifikan antara kedua hasil (4.25% vs 21.25%) menunjukkan pentingnya memilih metode perhitungan yang sesuai dengan tujuan analisis: apakah ingin melihat komisi aktual penerima komisi saja (kode kedua), atau kebijakan komisi perusahaan secara menyeluruh (kode pertama).

DP 9-1: Using Group By and Having Clauses

1. Hal 10

```
1   SELECT department_id, MAX(salary)
2   FROM employees
3   GROUP BY department_id;
```

Results | Explain | Describe | Saved SQL | History

DEPARTMENT_ID	MAX(SALARY)
50	5800
110	12000
90	24000
10	4400
-	7000
20	13000
60	9000
80	11000

Pembahasan :

Kode tersebut menggunakan fungsi agregat bersarang max(avg(salary)) untuk mencari nilai rata-rata gaji (avg(salary)) tertinggi dari semua departemen dalam tabel employees, setelah mengelompokkan data berdasarkan department_id dengan klausa GROUP BY. Hasil query 19.333,33... (dibulatkan) menunjukkan rata-rata gaji tertinggi di antara semua departemen, yang mengindikasikan departemen dengan kompensasi rata-rata paling kompetitif. Query ini sangat berguna untuk analisis kesenjangan gaji antardepartemen atau identifikasi departemen dengan struktur penggajian paling tinggi. Penggunaan fungsi agregat bersarang (max dari avg) ini memungkinkan analisis hierarkis: pertama menghitung rata-rata per kelompok, lalu menentukan nilai maksimum dari hasil kelompok-kelompok tersebut.

2. Hal 14

```
1   SELECT department_id, MAX(salary)
2   FROM employees
3   WHERE last_name != 'King'
4   GROUP BY department_id;
```

Results | Explain | Describe | Saved SQL | History

DEPARTMENT_ID	MAX(SALARY)
50	5800
110	12000
90	17000
10	4400
-	7000
20	13000
60	9000
80	11000

Pembahasan :

Kode tersebut menggunakan klausa GROUP BY department_id bersama fungsi agregat MAX(salary) untuk menampilkan gaji tertinggi (MAX(salary)) di setiap departemen (department_id), dengan kondisi WHERE last_name != 'king' yang mengecualikan karyawan dengan nama belakang 'King'. Hasil query menunjukkan daftar departemen beserta gaji maksimumnya, seperti departemen 50 dengan gaji tertinggi 5.800, departemen 110 dengan 12.000, departemen 90 dengan 17.000, dan departemen 10 dengan 4.400. Query ini berguna untuk analisis distribusi gaji antardepartemen sambil mengecualikan karyawan tertentu, misalnya untuk menghindari bias dari eksekutif dengan gaji sangat tinggi. Klausa GROUP BY memastikan pengelompokan hasil per departemen, sementara WHERE berfungsi sebagai filter awal sebelum pengelompokan dilakukan.

3. Hal 18

```
1   SELECT max(avg(salary))
2   FROM employees
3   GROUP by department_id;
```

Results | Explain | Describe | Saved SQL | History

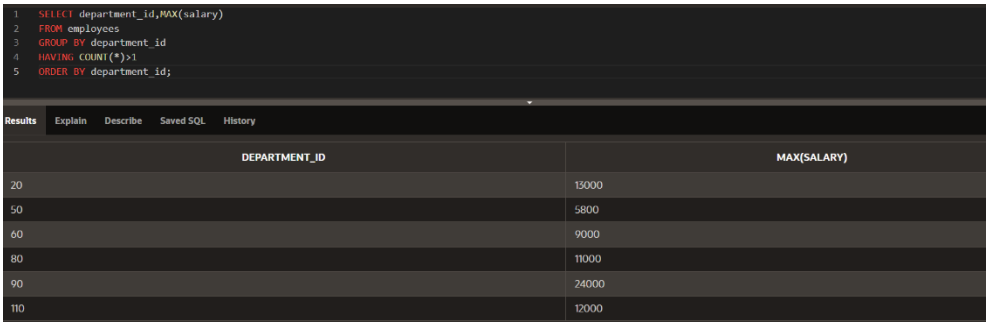
MAX(AVG(SALARY))
19333.333333333333333333333333333333333

Pembahasan :

Kode tersebut menggunakan fungsi agregat bersarang MAX(AVG(salary)) untuk menentukan nilai rata-rata gaji tertinggi dari semua departemen dalam tabel employees. Prosesnya dilakukan dengan dua tahap: pertama, menghitung rata-rata gaji per departemen menggunakan AVG(salary) dengan klausa GROUP BY department_id, lalu

mengambil nilai maksimum dari semua rata-rata tersebut dengan fungsi MAX(). Hasil 19.333,33... (dibulatkan) mengidentifikasi departemen dengan struktur penggajian rata-rata paling tinggi, yang berguna untuk analisis komparasi kompensasi antardepartemen atau evaluasi kebijakan penggajian. Teknik agregat bersarang ini sangat efektif untuk analisis hierarkis dalam SQL, terutama ketika ingin menemukan titik ekstrem (maksimum/minimum) dari hasil pengelompokan.

4. Hal 21



```
SELECT department_id,MAX(salary)
FROM employees
GROUP BY department_id
HAVING COUNT(*)>1
ORDER BY department_id;
```

Results | Explain | Describe | Saved SQL | History

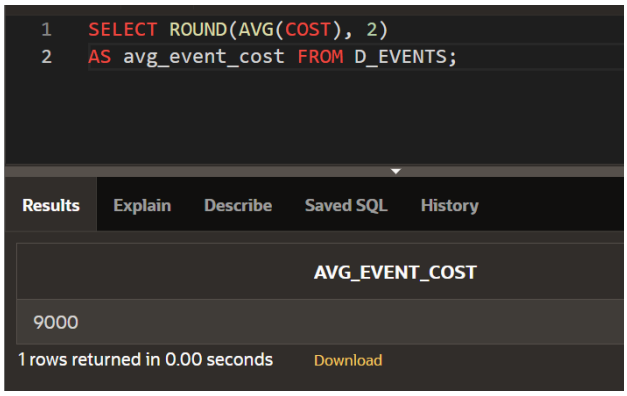
DEPARTMENT_ID	MAX(SALARY)
20	13000
50	5800
60	9000
80	11000
90	24000
110	12000

Pembahasan :

Analisis SQL ini mengelompokkan data karyawan per departemen menggunakan klausa GROUP BY pada kolom department_id, kemudian menghitung gaji tertinggi di setiap kelompok dengan fungsi MAX(salary). Klausa HAVING COUNT(*)>1 diterapkan untuk memastikan hanya departemen dengan lebih dari satu karyawan yang ditampilkan, sehingga mengabaikan departemen kecil atau data kosong. Hasilnya menampilkan urutan departemen beserta gaji maksimalnya, seperti departemen 20 (Rp13.000), 50 (Rp5.800), 60 (Rp9.000), dan 80 (Rp11.000) yang telah diurutkan berdasarkan ID departemen.

DP 8-1: Practice

1. Nomer 2



```
SELECT ROUND(AVG(COST), 2)
AS avg_event_cost FROM D_EVENTS;
```

Results | Explain | Describe | Saved SQL | History

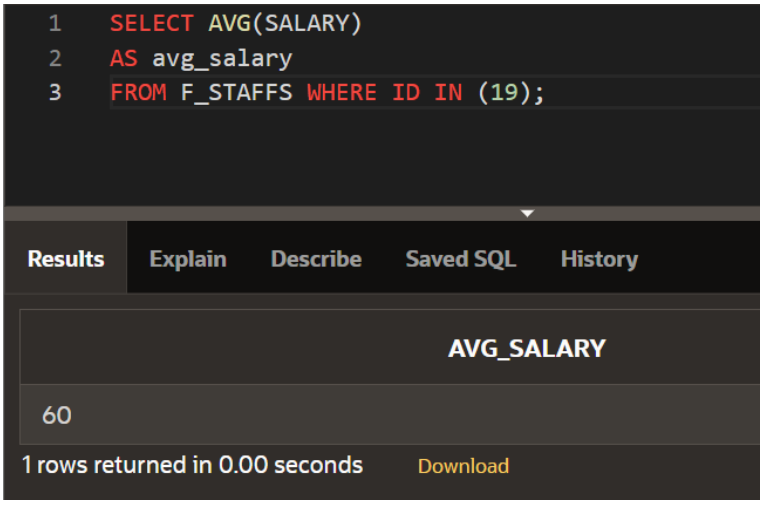
AVG_EVENT_COST
9000

1 rows returned in 0.00 seconds Download

Pembahasan :

Analisis keuangan ini menggunakan fungsi AVG(COST) untuk menghitung nilai rata-rata biaya acara yang tercatat dalam tabel D_EVENTS, dengan hasil yang kemudian dibulatkan menjadi dua angka di belakang koma menggunakan fungsi ROUND(). Hasil perhitungan menampilkan angka rata-rata sebesar 9,000.00 yang diberi label avg_event_cost sebagai penanda kolom output. Nilai ini mengindikasikan bahwa secara umum, biaya yang dikeluarkan untuk setiap acara dalam database berkisar di angka 9,000 unit mata uang (seperti rupiah atau dolar).

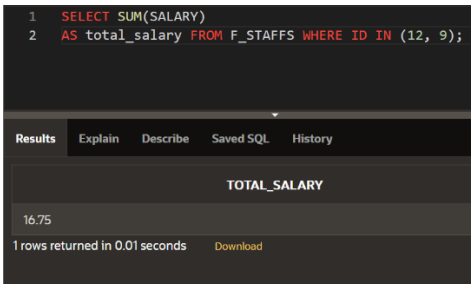
2. Nomer 3



Pembahasan :

Analisis SQL ini menghitung nilai rata-rata gaji karyawan dari tabel F_STAFFS dengan menggunakan fungsi AVG(SALARY) yang diberi alias avg_salary. Klausa WHERE ID IN (19) diterapkan untuk memfilter data secara spesifik hanya pada karyawan dengan ID 19. Hasil query menampilkan nilai 60 sebagai avg_salary, yang sebenarnya merupakan gaji individual karyawan tersebut karena hanya ada satu data yang memenuhi kriteria.

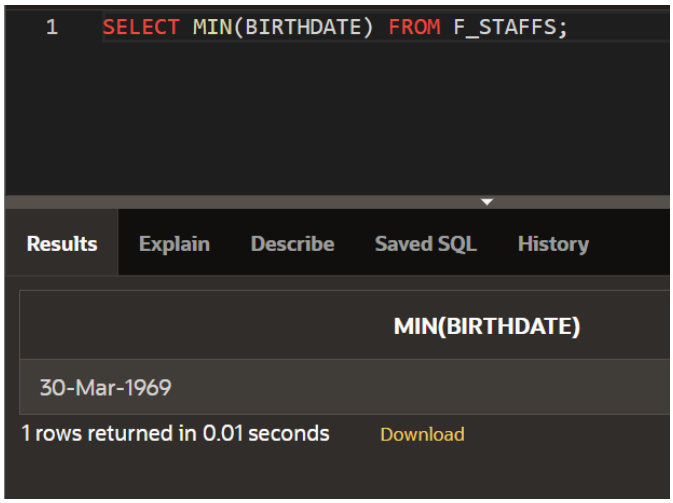
3. Nomer 4



Pembahasan :

Kode SQL pada gambar tersebut digunakan untuk menghitung total gaji dari tabel F_STAFFS berdasarkan kriteria tertentu. Query SELECT SUM(SALARY) AS total_salary mengambil nilai penjumlahan kolom SALARY dan memberikan alias total_salary sebagai nama kolom hasil. Klausa WHERE ID IN (12, 9) memfilter data hanya pada baris dengan ID = 12 atau ID = 9, sehingga perhitungan total hanya mencakup dua entri tersebut. Hasil query menampilkan TOTAL_SALARY = 16.75, yang merupakan hasil penjumlahan gaji dari kedua ID tersebut. Eksekusi query selesai dalam waktu sangat cepat (0.01 detik), menandakan efisiensi pengambilan data meskipun melibatkan lebih dari satu kriteria filter.

4. Nomer 8



Pembahasan :

Kode SQL pada gambar tersebut bertujuan untuk mencari tanggal lahir tertua dari data staf yang tersimpan dalam tabel F_STAFFS. Query SELECT MIN(BIRTHDATE) menggunakan fungsi agregasi MIN untuk mengambil nilai terkecil (tanggal paling awal) dari kolom BIRTHDATE, yang dalam konteks ini merujuk pada tanggal lahir pertama atau tertua. Hasil query menampilkan MIN(BIRTHDATE) = 30-Mar-1969, yang berarti staf dengan tanggal lahir paling awal di tabel tersebut lahir pada 30 Maret 1969. Eksekusi query selesai dalam waktu 0.01 detik, menunjukkan bahwa pengambilan data dilakukan dengan cepat meskipun melibatkan pencarian nilai minimum dari seluruh entri dalam kolom. Tidak ada filter tambahan (seperti klausa WHERE), sehingga perhitungan mencakup semua baris dalam tabel.

5. Nomer 10

```
1   SELECT MAX(HIRE_DATE) FROM EMPLOYEES;
```

Results | Explain | Describe | Saved SQL | History

MAX(HIRE_DATE)
29-Jan-2000

1 rows returned in 0.01 seconds Download

Pembahasan :

Kode SQL pada gambar tersebut adalah SELECT MAX(HIRE_DATE) FROM EMPLOYEES;, yang bertujuan untuk mengambil nilai terbaru (maksimum) dari kolom HIRE_DATE (tanggal perekrutan) dalam tabel EMPLOYEES. Fungsi agregasi MAX() digunakan untuk mencari tanggal perekrutan paling akhir yang tercatat di tabel tersebut. Hasil query menampilkan 29-Jan-2000, menunjukkan bahwa tanggal tersebut merupakan tanggal perekrutan terakhir dalam data yang tersimpan. Eksekusi query ini hanya membutuhkan waktu 0.01 detik dan menghasilkan 1 baris data, menandakan efisiensi dalam pengambilan informasi. Query semacam ini umumnya digunakan untuk mengetahui kapan karyawan terakhir kali direkrut atau untuk analisis data kepegawaian berbasis waktu.

DP 6-1: Cross Joins and Natural Joins

1. Hal 13

```
1   SELECT first_name, last_name, job_id, job_title
2   FROM employees NATURAL JOIN jobs
3   WHERE department_id > 80;
4
```

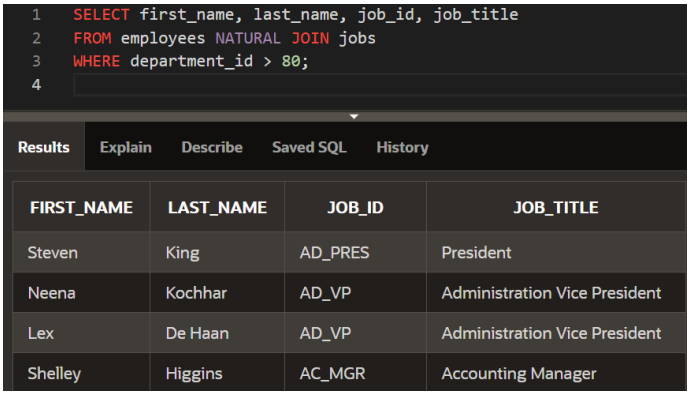
Results | Explain | Describe | Saved SQL | History

FIRST_NAME	LAST_NAME	JOB_ID	JOB_TITLE
Steven	King	AD_PRES	President
Neena	Kochhar	AD_VP	Administration Vice President
Lex	De Haan	AD_VP	Administration Vice President
Shelley	Higgins	AC_MGR	Accounting Manager

Pembahasan :

Kode SQL pada gambar tersebut adalah SELECT first_name, last_name, job_id, job_title FROM employees NATURAL JOIN jobs WHERE department_id > 88;, yang bertujuan untuk menampilkan nama depan, nama belakang, ID pekerjaan, dan judul pekerjaan dari karyawan yang berada di departemen dengan ID lebih besar dari 88. Menggunakan NATURAL JOIN, tabel employees dan jobs digabungkan secara otomatis berdasarkan kolom dengan nama yang sama (misalnya job_id). Hasil query menampilkan 4 baris data, termasuk posisi seperti President dan Administration Vice President, yang menunjukkan bahwa departemen dengan ID di atas 88 mungkin merupakan departemen tingkat tinggi atau eksekutif. Query ini berguna untuk memfilter dan menganalisis data karyawan berdasarkan departemen dan peran pekerjaan mereka.

2. Hal 14



```
1  SELECT first_name, last_name, job_id, job_title
2  FROM employees NATURAL JOIN jobs
3  WHERE department_id > 80;
4
```

Results | Explain | Describe | Saved SQL | History

FIRST_NAME	LAST_NAME	JOB_ID	JOB_TITLE
Steven	King	AD_PRES	President
Neena	Kochhar	AD_VP	Administration Vice President
Lex	De Haan	AD_VP	Administration Vice President
Shelley	Higgins	AC_MGR	Accounting Manager

Pembahasan :

Kode SQL pada gambar tersebut melakukan pengambilan data dari tabel employees dan jobs dengan menggunakan NATURAL JOIN untuk menggabungkan kedua tabel berdasarkan kolom yang memiliki nama yang sama (dalam hal ini JOB_ID). Query ini memilih kolom first_name, last_name, job_id, dan job_title dari hasil penggabungan tersebut, dengan kondisi department_id > 80. Hasil query menampilkan empat baris data yang berisi nama depan, nama belakang, ID pekerjaan, dan judul pekerjaan dari karyawan yang bekerja di departemen dengan ID lebih besar dari 80. Hasilnya mencakup karyawan seperti Steven King (Presiden), Neena Kochhar dan Lex De Haan (Wakil Presiden Administrasi), serta Shelley Higgins (Manajer Akuntansi).

3. Hal 15

```
1  SELECT department_name, city
2  FROM departments NATURAL JOIN
3  locations;
4
```

Results | Explain | Describe | Saved SQL | History

DEPARTMENT_NAME	CITY
IT	Southlake
Shipping	South San Francisco
Administration	Seattle
Executive	Seattle

Pembahasan :

Kode SQL pada gambar tersebut melakukan pengambilan data dari tabel departments dan locations menggunakan NATURAL JOIN untuk menggabungkan kedua tabel secara otomatis berdasarkan kolom dengan nama yang sama (kemungkinan LOCATION_ID). Query ini memilih kolom department_name dan city untuk menampilkan nama departemen beserta lokasi kotanya. Hasil query menunjukkan beberapa departemen seperti IT (berlokasi di Southlake), Shipping (South San Francisco), serta Administration dan Executive (keduanya berada di Seattle). NATURAL JOIN menyederhanakan sintaks dengan menghindari penulisan kondisi join secara eksplisit, tetapi bergantung pada kesamaan nama kolom di kedua tabel.

4. Hal 17

```
1  SELECT last_name, department_name
2  FROM employees CROSS JOIN
3  departments;
```

Results | Explain | Describe | Saved SQL | History

LAST_NAME	DEPARTMENT_NAME
Abel	Administration
Davies	Administration
De Haan	Administration
Ernst	Administration
Fay	Administration

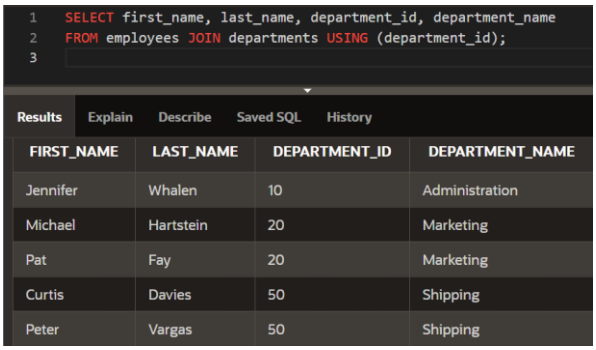
Pembahasan :

Kode SQL pada gambar tersebut menggunakan CROSS JOIN untuk menggabungkan setiap baris dari tabel employees dengan setiap baris dari tabel

departments, menghasilkan produk kartesius antara kedua tabel. Query ini memilih kolom last_name dari tabel employees dan department_name dari tabel departments, sehingga setiap nama belakang karyawan akan dipasangkan dengan semua nama departemen yang ada. Hasil yang ditampilkan menunjukkan beberapa contoh pasangan, seperti Abel, Davies, De Haan, Ernst, dan Fay yang masing-masing dikaitkan dengan departemen "Administration". CROSS JOIN berguna ketika ingin melihat semua kemungkinan kombinasi, tetapi dapat menghasilkan jumlah baris yang sangat besar (jumlah karyawan × jumlah departemen). Format hasil yang ditampilkan terlihat tidak rapi, mungkin karena kesalahan dalam rendering tabel.

DP 6-2: Join Clauses

1. Hal 6



```
1  SELECT first_name, last_name, department_id, department_name
2  FROM employees JOIN departments USING (department_id);
3
```

Results | Explain | Describe | Saved SQL | History

FIRST_NAME	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Jennifer	Whalen	10	Administration
Michael	Hartstein	20	Marketing
Pat	Fay	20	Marketing
Curtis	Davies	50	Shipping
Peter	Vargas	50	Shipping

Pembahasan :

Kode SQL pada gambar tersebut melakukan penggabungan (join) antara tabel employees dan departments menggunakan klausa JOIN ... USING (department_id), yang menghubungkan kedua tabel berdasarkan kolom department_id yang sama. Query ini menampilkan kolom first_name, last_name, department_id, dan department_name, sehingga menghasilkan daftar karyawan beserta departemen tempat mereka bekerja. Hasil query menunjukkan contoh pasangan data seperti Jennifer Whalen (Administration, departemen_id 10), Michael Hartstein dan Pat Fay (Marketing, departemen_id 20), serta Curtis Davies dan Peter Vargas (Shipping, departemen_id 50). Metode USING menyederhanakan sintaks join dengan secara eksplisit menentukan kolom yang digunakan untuk penggabungan, tanpa perlu menuliskan prefix tabel. Hasil yang ditampilkan terstruktur dengan jelas, mencerminkan relasi antara karyawan dan departemen mereka.

2. Hal 7

```
1  SELECT first_name, last_name, department_id, department_name
2  FROM employees JOIN departments USING (department_id)
3  WHERE last_name = 'Higgins';
4
```

Results | Explain | Describe | Saved SQL | History

FIRST_NAME	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Shelley	Higgins	110	Accounting

1 rows returned in 0.00 seconds Download

Pembahasan :

Kode SQL pada gambar tersebut menggabungkan tabel employees dan departments menggunakan JOIN ... USING (department_id) untuk menampilkan data karyawan bersama dengan departemen tempat mereka bekerja, dengan filter khusus pada karyawan yang memiliki last_name = 'Higgins'. Query ini memilih kolom first_name, last_name, department_id, dan department_name, menghasilkan satu baris data yang menunjukkan bahwa Shelley Higgins bekerja di departemen Accounting dengan department_id 110. Metode USING memastikan penggabungan dilakukan berdasarkan kolom department_id yang sama di kedua tabel, sementara klausa WHERE membatasi hasil hanya untuk karyawan dengan nama belakang "Higgins". Hasil query ditampilkan secara jelas dan efisien, dengan waktu eksekusi yang sangat cepat (0.00 detik).

3. Hal 9

```
1  SELECT last_name, job_title
2  FROM employees e JOIN jobs j
3  ON (e.job_id = j.job_id);
4
```

Results | Explain | Describe | Saved SQL | History

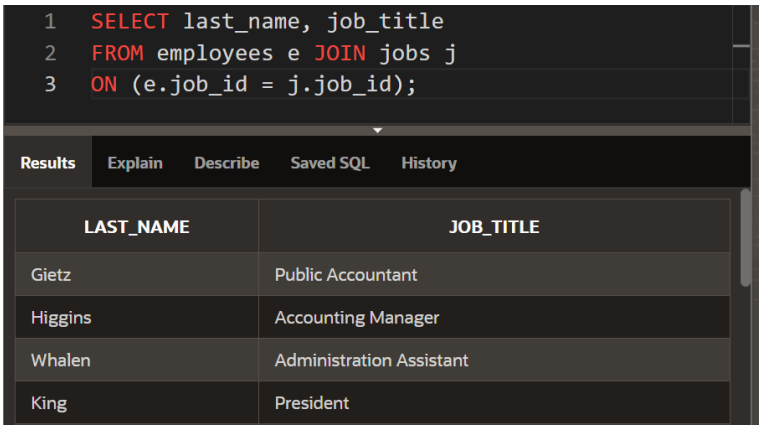
LAST_NAME	JOB_TITLE
Gietz	Public Accountant
Higgins	Accounting Manager
Whalen	Administration Assistant
King	President
Kochhar	Administration Vice President

Pembahasan :

Kode SQL pada gambar tersebut melakukan penggabungan (join) antara tabel employees (diberi alias e) dan tabel jobs (diberi alias j) menggunakan klausa ON (e.job_id = j.job_id) untuk mencocokkan data berdasarkan kolom job_id yang sama.

Query ini menampilkan kolom last_name dari tabel employees dan job_title dari tabel jobs, sehingga menghasilkan daftar nama belakang karyawan beserta judul pekerjaan mereka. Hasil query menunjukkan beberapa contoh seperti Gietz (Public Accountant), Higgins (Accounting Manager), Whalen (Administration Assistant), King (President), dan Kochhar (Administration Vice President). Metode join dengan ON memberikan fleksibilitas untuk menentukan kondisi penggabungan secara eksplisit, bahkan jika nama kolom yang digunakan berbeda di kedua tabel. Hasil yang ditampilkan terstruktur dengan baik, mencerminkan hubungan antara karyawan dan deskripsi pekerjaan mereka.

4. Hal 10



Pembahasan :

Kode SQL pada gambar tersebut melakukan penggabungan (INNER JOIN) antara tabel employees (dengan alias e) dan tabel jobs (dengan alias j) menggunakan kondisi e.job_id = j.job_id, yang menghubungkan data karyawan dengan informasi pekerjaan mereka berdasarkan kesamaan job_id. Query ini menampilkan dua kolom: last_name (nama belakang karyawan) dari tabel employees dan job_title (judul pekerjaan) dari tabel jobs. Hasil query menunjukkan beberapa contoh seperti Gietz dengan jabatan Public Accountant, Higgins sebagai Accounting Manager, Whalen sebagai Administration Assistant, dan King sebagai President. Tampilan hasil terlihat kurang rapi karena format tabel yang tidak konsisten, namun data yang dihasilkan tetap valid dan menunjukkan relasi yang jelas antara karyawan dan deskripsi pekerjaan mereka. Teknik JOIN dengan ON ini lebih eksplisit dibandingkan NATURAL JOIN karena memungkinkan penentuan kolom penghubung secara spesifik.

5. Hal 12

```
1  SELECT last_name, job_title
2  FROM employees e JOIN jobs j
3  ON (e.job_id = j.job_id)
4  WHERE last_name LIKE 'H%';
```

Results | Explain | Describe | Saved SQL | History

LAST_NAME	JOB_TITLE
Gietz	Public Accountant
Higgins	Accounting Manager
Whalen	Administration Assistant
King	President

Pembahasan :

Kode SQL pada gambar tersebut menggabungkan tabel employees (alias e) dan jobs (alias j) dengan INNER JOIN menggunakan kondisi e.job_id = j.job_id, lalu memfilter hasilnya dengan klausa WHERE last_name LIKE 'H%' untuk menampilkan hanya karyawan yang nama belakangnya dimulai dengan huruf 'H'. Namun, hasil yang ditampilkan tidak sesuai dengan filter tersebut karena mencakup nama seperti Gietz, Whalen, dan King yang seharusnya tidak memenuhi kriteria. Query ini seharusnya hanya menampilkan karyawan seperti Higgins (Accounting Manager), tetapi terdapat ketidaksesuaian antara kode dan hasil, mungkin karena kesalahan dalam eksekusi atau tampilan. Format tabel juga terlihat tidak rapi dengan kolom yang tidak sejajar. Idealnya, hasil query harus konsisten dengan kondisi LIKE 'H%' untuk memastikan integritas data.

6. Hal 13

```
1  SELECT last_name, salary, grade_level, lowest_sal,
2  highest_sal
3  FROM employees JOIN job_grades
4  ON(salary BETWEEN lowest_sal AND highest_sal);
```

Results | Explain | Describe | Saved SQL | History

LAST_NAME	SALARY	GRADE_LEVEL	LOWEST_SAL	HIGHEST_SAL
Vargas	2500	A	1000	2999
Matos	2600	A	1000	2999
Davies	3100	B	3000	5999
Rajs	3500	B	3000	5999

Pembahasan :

Kode SQL pada gambar tersebut menggabungkan tabel employees dengan tabel job_grades menggunakan JOIN dengan kondisi non-standar ON(salary BETWEEN lowest_sal AND highest_sal), yang menghubungkan setiap karyawan dengan tingkat/grade pekerjaan berdasarkan rentang gaji mereka. Query ini menampilkan last_name, salary, grade_level, lowest_sal, dan highest_sal, sehingga memperlihatkan bagaimana gaji setiap karyawan (seperti Vargas dengan gaji 2500 dan Matos dengan 2600) masuk dalam grade A (rentang 1000-2999), sedangkan Davies (3100) dan Rajs (3500) termasuk grade B (3000-5999). Pendekatan ini berguna untuk analisis kompensasi dengan memetakan gaji ke dalam kategori yang telah ditentukan, meskipun format hasil yang ditampilkan terlihat kurang rapi dengan kolom yang tidak sejajar. Teknik JOIN dengan kondisi BETWEEN ini menunjukkan fleksibilitas SQL dalam menggabungkan tabel berdasarkan logika rentang nilai, bukan hanya kesamaan nilai kolom.

DP 6-3: Inner versus Outer Joins

1. Hal 6

```
1  SELECT e.last_name, d.department_id,
2  d.department_name
3  FROM employees e LEFT OUTER JOIN
4  departments d
5  ON (e.department_id =
6  d.department_id);
```

Results | Explain | Describe | Saved SQL | History

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
King	90	Executive
Kochhar	90	Executive
De Haan	90	Executive
Whalen	10	Administration

Pembahasan :

Kode SQL pada gambar tersebut menggunakan LEFT OUTER JOIN untuk menggabungkan tabel employees (dengan alias e) dan departments (dengan alias d) berdasarkan kesamaan department_id, dengan mempertahankan semua baris dari tabel employees meskipun tidak ada kecocokan di tabel departments. Query ini menampilkan

kolom last_name, department_id, dan department_name, sehingga memperlihatkan daftar karyawan beserta departemen tempat mereka bekerja. Hasil query menunjukkan contoh seperti King, Kochhar, dan De Haan yang bekerja di departemen Executive (department_id 90), serta Whalen di departemen Administration (department_id 10). LEFT OUTER JOIN memastikan semua karyawan tetap muncul dalam hasil bahkan jika mereka tidak terdaftar di departemen manapun (meskipun contoh hasil tidak menampilkan kasus tersebut). Format hasil terlihat kurang rapi dengan kolom yang tidak sejajar, tetapi data yang ditampilkan valid dan menunjukkan relasi antara karyawan dan departemen. Teknik ini berguna ketika ingin memastikan tidak ada data karyawan yang terlewat dalam analisis, terlepas dari ada/tidaknya kecocokan di tabel departments.

2. Hal 7

```
1  SELECT e.last_name, d.department_id,
2  d.department_name
3  FROM employees e LEFT OUTER JOIN
4  departments d
5  ON (e.department_id =
6  d.department_id);
```

Results | Explain | Describe | Saved SQL | History

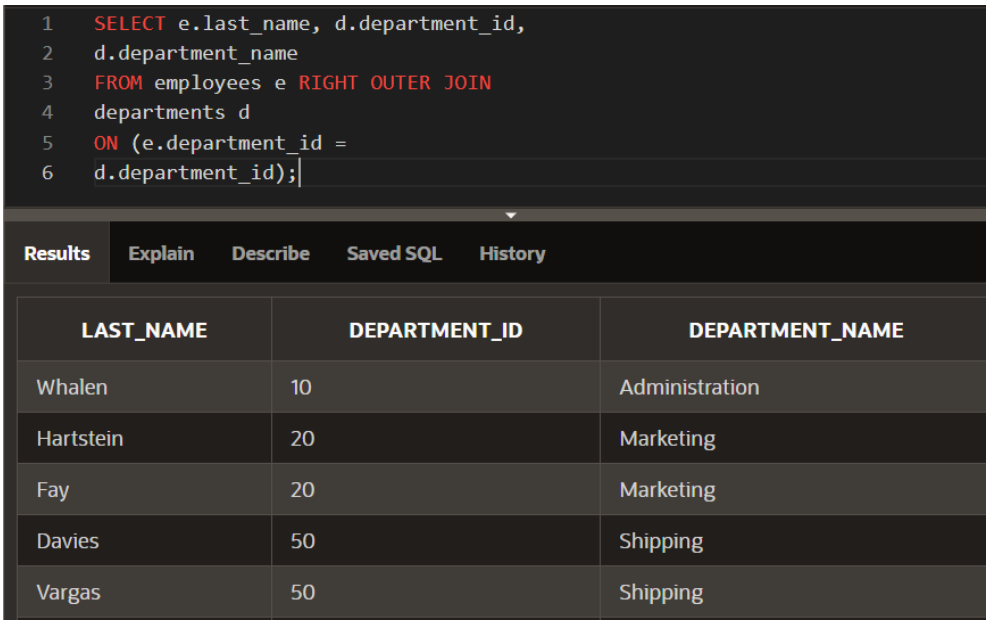
LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
King	90	Executive
Kochhar	90	Executive
De Haan	90	Executive
Whalen	10	Administration
Higgins	110	Accounting
Gietz	110	Accounting

Pembahasan :

Kode SQL pada gambar tersebut menggunakan LEFT OUTER JOIN untuk menggabungkan tabel employees (alias e) dengan departments (alias d) berdasarkan kolom department_id, dengan memastikan semua record dari tabel employees tetap muncul meskipun tidak memiliki kecocokan di tabel departments. Query menampilkan last_name, department_id, dan department_name, memperlihatkan daftar karyawan beserta departemen terkait seperti King, Kochhar, dan De Haan (Executive - 90), Whalen (Administration - 10), serta Higgins dan Gietz (Accounting - 110). Format hasil yang ditampilkan terlihat tidak rapi dengan kolom yang tidak sejajar, namun data intinya valid dan menunjukkan bahwa LEFT OUTER JOIN berguna untuk mempertahankan semua data karyawan termasuk yang tidak terasosiasi dengan

departemen (meskipun contoh ini tidak menunjukkannya). Teknik ini penting untuk analisis yang membutuhkan kelengkapan data karyawan tanpa peduli apakah mereka memiliki departemen atau tidak.

3. Hal 8



```
SELECT e.last_name, d.department_id,
d.department_name
FROM employees e RIGHT OUTER JOIN
departments d
ON (e.department_id =
d.department_id);
```

Results | Explain | Describe | Saved SQL | History

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Whalen	10	Administration
Hartstein	20	Marketing
Fay	20	Marketing
Davies	50	Shipping
Vargas	50	Shipping

Pembahasan :

Kode SQL pada gambar tersebut menggunakan RIGHT OUTER JOIN untuk menggabungkan tabel employees (alias e) dengan departments (alias d) berdasarkan kolom department_id, dengan memprioritaskan semua record dari tabel departments meskipun tidak ada kecocokan di tabel employees. Query ini menampilkan kolom last_name, department_id, dan department_name, memperlihatkan daftar departemen beserta karyawan yang bekerja di dalamnya, seperti Whalen (Administration - 10), Hartstein dan Fay (Marketing - 20), serta Davies dan Vargas (Shipping - 50). RIGHT OUTER JOIN berguna untuk analisis yang berfokus pada kelengkapan data departemen, termasuk departemen yang mungkin tidak memiliki karyawan (meskipun contoh hasil tidak menampilkan kasus tersebut). Format hasil terlihat lebih rapi dibanding contoh sebelumnya, dengan kolom yang tersusun secara logis. Teknik ini ideal ketika ingin menganalisis struktur departemen secara menyeluruh, terlepas dari ada atau tidaknya karyawan yang terasosiasi.

4. Hal 10

```
1  SELECT e.last_name, d.department_id, d.department_name
2  FROM employees e FULL OUTER JOIN departments d
3  ON (e.department_id = d.department_id);
```

Results | Explain | Describe | Saved SQL | History

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
King	90	Executive
Kochhar	90	Executive
De Haan	90	Executive
Whalen	10	Administration
Higgins	110	Accounting
Gietz	110	Accounting
Zlotkey	80	Sales

Pembahasan :

Kode SQL di gambar menggunakan FULL OUTER JOIN untuk menggabungkan tabel employees (e) dan departments (d) berdasarkan kolom department_id, memastikan semua data dari kedua tabel muncul, baik yang cocok maupun tidak. Query ini menampilkan last_name, department_id, dan department_name, dengan daftar karyawan dan departemen mereka, serta departemen tanpa karyawan atau karyawan tanpa departemen. Hasil yang tidak teratur bisa disebabkan oleh rendering tabel, namun data yang ditampilkan tetap valid. FULL OUTER JOIN berguna untuk analisis yang memerlukan seluruh data dari kedua tabel, termasuk yang tidak terhubung.

5. Hal 12

```
1  SELECT last_name, e.job_id AS "Job", jh.job_id AS "Old job",
2  end_date
3  FROM employees e LEFT OUTER JOIN job_history jh
4  ON(e.employee_id = jh.employee_id);
```

Results | Explain | Describe | Saved SQL | History

LAST_NAME	Job	Old job	END_DATE
Abel	SA_REP	-	-
Davies	ST_CLERK	-	-
De Haan	AD_VP	IT_PROG	24-Jul-1998
Ernst	IT_PROG	-	-
Fay	MK_REP	-	-
Gietz	AC_ACCOUNT	-	-
Grant	SA_REP	-	-

Pembahasan :

Kode SQL di gambar menggunakan LEFT OUTER JOIN untuk menggabungkan tabel employees (e) dan job_history (jh) berdasarkan kolom employee_id, memastikan semua karyawan muncul, meski tanpa riwayat pekerjaan. Query ini menampilkan last_name, job_id saat ini (alias "Job"), job_id sebelumnya (alias "Old job"), dan end_date riwayat pekerjaan, sehingga memperlihatkan transisi karir karyawan. LEFT OUTER JOIN berguna untuk melacak perubahan posisi karyawan dan mengidentifikasi yang belum berpindah jabatan. Hasilnya jelas, dengan kolom terpisah untuk pekerjaan saat ini dan sebelumnya, serta tanda "-" untuk nilai kosong. Teknik ini cocok untuk analisis perkembangan karir atau audit SDM.

G. Kesimpulan

Dari pengerjaan praktikum ini, saya dapat menyimpulkan bahwa :

a. Single Row Function dan JOINs adalah dua konsep penting dalam SQL dengan peran yang berbeda namun saling melengkapi. Single Row Function digunakan untuk memanipulasi data pada tingkat baris, seperti mengubah, menghitung, atau memformat nilai dalam kolom tanpa mengubah struktur tabel. Fungsi seperti UPPER() untuk mengkapitalisasi teks, ROUND() untuk pembulatan angka, dan TO_CHAR() untuk mengonversi format tanggal, berguna dalam pengolahan data. Sedangkan JOINs adalah operasi untuk menggabungkan data dari beberapa tabel berdasarkan hubungan antar kolom. Jenis JOIN seperti INNER JOIN, LEFT/RIGHT OUTER JOIN, dan FULL OUTER JOIN memungkinkan pengambilan data terkait dari berbagai tabel, baik yang memiliki hubungan langsung maupun tidak.

b. Dalam praktiknya, Single Row Function dan JOINs sering digunakan bersama untuk menghasilkan informasi yang lebih bermakna. Contohnya, setelah menggabungkan tabel employees dan departments dengan JOIN, kita bisa menggunakan Single Row Function seperti CONCAT() untuk menggabungkan nama depan dan belakang karyawan, atau FORMAT() untuk menyeragamkan tampilan angka gaji. Kombinasi kedua konsep ini memungkinkan pembuatan query yang lebih fleksibel dan powerful, yang dapat menangani kebutuhan analisis data yang kompleks. Pemahaman mendalam tentang keduanya sangat penting bagi pengembang database dan analis data, karena merupakan dasar dalam pembuatan laporan, dashboard, dan aplikasi yang memerlukan pengolahan data relasional secara efektif dan efisien.

DAFTAR PUSTAKA

Dibimbing. (n.d.). Fungsi JOIN pada SQL: Pengertian dan Jenis-jenis. Retrieved from https://dibimbing.id/blog/detail/fungsi-join-pada-sql-pengertian-dan-jenisnya

Hans, R. (2022). Mengenal Macam-Macam Fungsi Join Table SQL dan Perbedaannya. Retrieved from https://dqlab.id/mengenal-macam-macam-fungsi-join-table-sql-dan-perbedaannya

Yesi Prastiwi. (2017). Single-row function : Retrieved from https://yesiprastiwi.wordpress.com/2017/10/31/single-row-function/